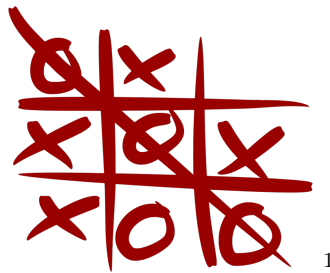


Introduction to computer networking

First part of the assignment

Academic year 2025-2026



Abstract

In this assignment, students will have to implement a client-server application using Java Sockets. The server is a game platform, i.e. it allows a user to play the game of Tic-tac-toe. The client will interact with the user to display the game status and place symbols.

Sections 1 and 2 define your assignment objectives. Sections 3 and 4 summarize the technologies we use. Section 5 describes the report. Section 6 presents the practical guidelines.

Students will work alone using the Java language. The **Hard Deadline** is October 26th, 2025.

1 Tic-tac-toe platform

As stated in the abstract, you will implement a Tic-tac-toe platform using Java Sockets. The server waits for TCP connections on a given port, receives requests through that connection when established, potentially updates its internal state accordingly, and responds with a formatted response following the custom-made Tic-tac-toe protocol.

The server will be up for new connections and be able to handle concurrency. Each client either plays its own game against a bot or plays against another player. The session scope is the TCP connection (identified by the client IP address and port).

¹Image taken from wikipedia.org: https://en.wikipedia.org/wiki/Tic-tac-toe#/media/File:Tic_tac_toe.svg

The client will interact with the user using terminal I/Os, send requests to the server, read its responses, and update its display accordingly.

Figure 1 and 2 show examples of a potential output from the client, interacting with the user and with the server. You are free to design your own user interface as long as the message exchange with the server respects the Tic-tac-toe Protocol (cfr. 2).

1.1 Rules

1.1.1 Overview

On game start, the server generates an empty 3×3 grid. This grid is 0-indexed from top-left to bottom-right. Players alternate placing symbols (X and O) on the grid; **X always plays first**. The objective is to complete a line of three of your own symbol horizontally, vertically, or diagonally.

Two game modes are available, playing against a bot and playing against another player. Once it is the user's turn, he can decide to either place his symbol on the grid, or request the grid status, or quit the game. The client software can help the user build its request and send it to the server. Upon receiving a request, the server updates the game status accordingly and sends back an appropriate response to the client.

The game ends either when a line of 3 symbols is achieved, or when the grid is full, or if the user(s) quit(s) prematurely.

1.1.2 Game mode selection

At the beginning of a new session, the user is prompted to select the desired game mode. Two options are available:

- **Single-player mode:** The user selects its symbol and plays against a bot. The bot acts as the opponent and alternates turns with the user. **It randomly places its symbol on available spots.**
- **Multiplayer mode:** The user plays against another human player. Both clients connect to the server, which coordinates the turns and ensures synchronization of the game state between them. Players may not decide their symbol, it is the server that will assign X to the player that joined the game first and O to the second.

1.1.3 Single-player mode

In this game mode, the player will not have to wait for another player to place its symbol as the server can immediately place the bot's symbol when it is its turn. The user can perform one of two actions:

- Place his/her symbol on the grid.

- Quit the game. The connection and resources must be properly released by the server.

Figure 1 shows an example of a single-player mode game. Notice that the bot symbol 'X', is immediately placed on the grid when it's its turn.

```

Welcome to Tictactoe
Select the game mode
1) Bot
2) Player
1
Choose your symbol X or O:
O
X

Place your symbol the grid: 'row' 'col'
Quit: q
0 1
XO
  X

Place your symbol the grid: 'row' 'col'
Quit: q
0 2
XOO
X X

Place your symbol the grid: 'row' 'col'
Quit: q
2 0
XOO
X X
OX

Place your symbol the grid: 'row' 'col'
Quit: q
2 2
XOO
XXX
OXO
X WON

```

Figure 1: Example of a potential console output in single player mode.

1.1.4 Multiplayer mode

In this game mode, the player may have to wait for the other player to place their symbol before continuing. For simplicity, the Tic-tac-toe Protocol follows a **client-pull paradigm**, where only the client can initiate communication by sending requests and the server only responds.

The user can perform the following actions:

- Place his symbol on the grid. If it is not his turn, the server will reject the request and return an error message.
- Request an update of the current state of the grid in order to check whether the other player has made a move.
- Quit the game. If only one player quits, the resources associated with that player are released, while any shared resources remain available for the other player.

Figure 2 shows an example of a multiplayer mode game. Notice that the player can try to place his symbol without first requesting an update. Moreover, the session is only terminated when the player quits.

<pre> Welcome to Tictactoe Select the game mode 1) Bot 2) Player 2 You are player X Place your symbol the grid: 'row' 'col' Update grid: u Quit: q 1 1 X Place your symbol the grid: 'row' 'col' Update grid: u Quit: q 0 0 CELL OCCUPIED Place your symbol the grid: 'row' 'col' Update grid: u Quit: q u 0 X Place your symbol the grid: 'row' 'col' Update grid: u Quit: q 0 1 OX X </pre>	<pre> Place your symbol the grid: 'row' 'col' Update grid: u Quit: q u OXO X Place your symbol the grid: 'row' 'col' Update grid: u Quit: q 2 1 OXO X X X WON Place your symbol the grid: 'row' 'col' Update grid: u Quit: q u OXO X X X WON Place your symbol the grid: 'row' 'col' Update grid: u Quit: q q </pre>
--	---

Figure 2: Example of a potential console output in multiplayer mode.

1.2 Launch

The software is invoked on the command line with no additional arguments:
`java TictactoeServer` for the server and `java TictactoeClient` for the client.

The server listens on port `2xxx` – where `xxx` are the last three digits of your ULiege student ID.

1.3 Incorrect or incomplete commands

When dealing with network connections, you can never assume that the other side will behave as you expect.

It is thus your responsibility to check the validity of the client's request (and respond with the appropriate error code if something goes wrong) and to ensure that a malevolent person won't make your server freeze by initiating a TCP connection and keeping it open for an indefinite period of time (see Section 3 for a counter-measure).

2 The Tic-tac-toe Protocol

The Tic-tac-toe Protocol (TP) is a custom-made protocol designed for this assignment only. This protocol is *text-oriented*, meaning that all exchanges are intended to be human-readable (as opposed to "binary" protocols such as IP, TCP and BitTorrent that feature custom information packing). This protocol is a multiline protocol, each request/response consists of a few lines of **uppercase** characters ending with "\r\n" (CRLF) and ends in an empty line (i.e. each request end with "\r\n\r\n").

Figure 3 is an example of a potential exchange between the different entities using the protocol. Note that the " " (white space) have been replaced with "_" for readability.

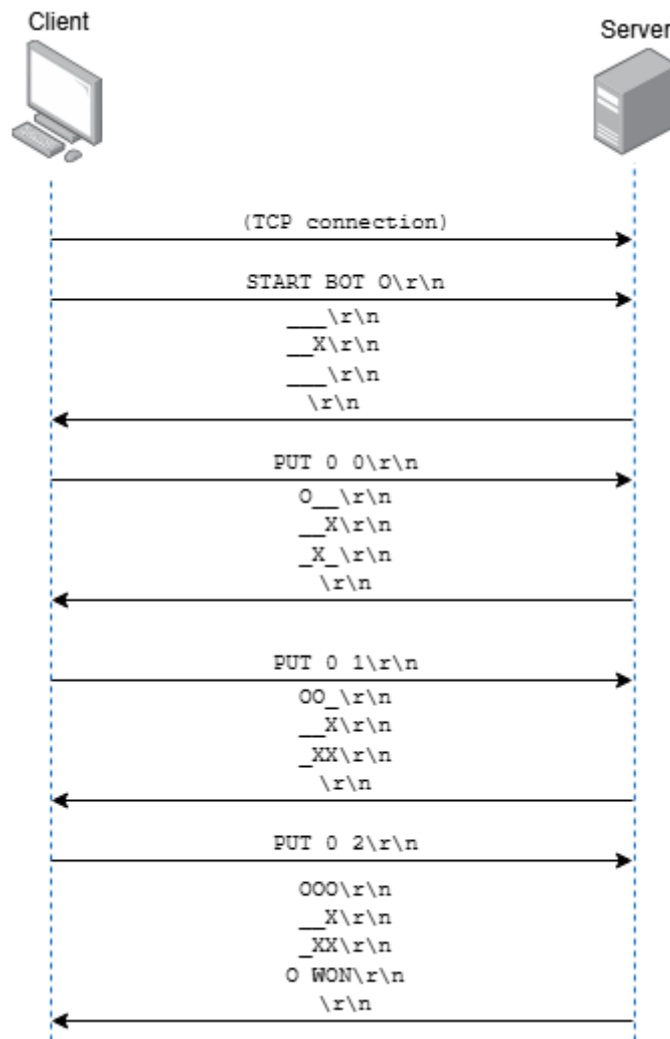


Figure 3: Example of the Tic-tac-toe Protocol.

2.1 Client requests

The client will either select the game mode, place his symbol, update the grid, or quit. It will thus send either:

- "START BOT *symbol*" or "START PLAYER", to respectively start the game in single-player mode or multiplayer mode. *symbol* must either be X or O and this request must be the first one
- "PUT *row col*" where *row* and *col* are **NUMBERS**, to place a symbol
- "UPDATE", to get the current state of the grid
- "QUIT", to signal to the server that it closes the connection

Note that all these requests end with "\r\n\r\n".

2.1.1 Server responses

Most responses contain a *grid* that has the following format:

- " " (white space), if the cell is empty.
- "X" (resp. "O"), if the cell contains "X" (resp. "O").
- Each line ends in "\r\n".

Moreover, a *GameStatus* message can be appended at the end of the grid. If someone won the game, *GameStatus* is "*symbol* WON". If the game ended in a draw, *GameStatus* is "DRAW". If a player quits in multiplayer mode before the end of the game, *GameStatus* is "OPPONENT QUIT"

When receiving "START BOT *symbol*", the games starts in single-player mode. The server responds with *grid*. If the player chooses the "O" symbol (player goes second), the bot player must play on the grid before the server respond with the first *grid*.

When receiving "START PLAYER", the games starts in multiplayer mode. The match making is simplistic, two consecutive players requesting to play a multiplayer game join the same one. The server responds with "PLAYER *symbol*" where *symbol* is "X" if the player joined the game first and "O" otherwise.

If the game hasn't started yet and a valid request is received, the server responds with "NO GAME MODE".

When receiving a "PUT" request, the response will depend on the cell that is selected and the player's turn. In normal circumstances, the server respond with an updated *grid* where the player's symbol is placed in *row col*.

Concerning edge cases:

- If the cell is not empty, the server responds with "CELL OCCUPIED".
- If it is not the player's turn, the server responds with "NOT YOUR TURN".
- If the *row* and/or *col* of a "PUT" request is out of range, the server respond with "INVALID RANGE".
- If the game is already finished, the game responds with "GAME FINISHED".
- If placing a symbol ends the game, *grid* + *GameStatus*.

When receiving an "UPDATE" request, the server responds with *grid* + *GameStatus*.

When receiving "QUIT", the server will close the connection to the client.

Finally, for any other request that it might receive, the server will answer with "WRONG" and keep the connection open.

3 Multi-Threading

A very simple solution for the server's code would be to accept a connection, handle the requests, then go back to accepting new connections, and so forth.

Now, imagine that a client behaves badly and sends an incomplete request. The server would then be blocked and would not be able to handle new connections.

A simple solution to handle this problem is to separate the code logic into separate threads. The main thread will loop on the `accept()` method and, when it succeeds, forwards the task to a new Thread. That way, even if one client is erroneous, it will only impact one of the server's Threads and other clients will be served correctly. In addition, using the method `setSoTimeout` on the socket will ensure that a client that stays silent for too long will trigger an exception at server-side.

We will see in the second part of the assignment that this method also has a flaw, but will be considered as acceptable for this first part.

4 Stream-oriented protocols (TCP)

The TP protocol relies on the TCP protocol, which is stream-oriented.

This has at least three implications:

1. You can be sure that the connection is lossless and that there won't be any packet re-ordering.

2. The sender process could be *buffering* the output, i.e. the data that is requested to be sent might not have been effectively sent through the socket yet. This can be solved by calling the `flush()` method on the `OutputStream`.
3. **The data requested to be sent with one call to a write method will not necessarily be received with only one read method from the other side.** Several reads could be necessary to recover the entire information. This is because we are reading from a stream, not messages (as we would with UDP, for instance). We thus have to find a way to delimit message boundaries on the stream. TP achieves this by having a delimiter (CRLF) after each request. In practice, you should thus use a class that takes multiple reads into account (such as `BufferedReader`) or perform multiple reads if using a low-level `read`.

5 Report

Your program will be completed with a .pdf report that follows the provided template and doesn't exceed 3 pages. The report will address the following points:

Software architecture: How have you broken down the problem to come to the solution? Name the major classes and their role.

Program Logic: How does your **server** work? Sequentially describe at a high level which operations it performs for accepting a connection to the end of a game. Limit yourself to a single-player game.

TCP stream: How did you **read** the requests from the TCP stream? You are allowed to provide the source code that handles this operation.

Multi-thread organisation: How did you achieve multi-threading in your program? You may also include source code to justify. Did you synchronize the activity of the different threads?

Robustness: How did you make your server robust? How did you manage the different types of exceptions that your server might receive?

6 Guidelines

- You will implement the programs using Java 1.17, with packages `java.lang`, `java.io`, `java.net` and `java.util`,
- You will ensure that your program can be terminated at any time simply using CTRL+C, and avoid the use of `ShutdownHooks`
- You will not manipulate any file on the local file system.

- You will ensure your main class is named TictactoeServer (resp. TictactoeClient), located in TictactoeServer.java (resp. TictactoeClient.java) at the **root of the archive**, and does not contain any **package** instruction. You are allowed to create more classes and java files than these two imposed ones.
- You will not cluster your program in packages or directories. All java files should be found in the same directory.
- You will ensure that your program is fully operational (i.e. compilation and execution). You are encouraged to test your program on the student's machines in the lab (ms8xx.montefiore.ulg.ac.be).

Submissions that do not observe these guidelines could be ignored during evaluation.

Your commented source code alongside the report will be delivered no later than 26th October 2025 (**Hard deadline**) to the Montefiore Submission platform (<https://submit.montefiore.uliege.be/>) as a .zip package.

7 Plagiarism

Plagiarism is the practice of taking someone else's work or ideas and passing them off as one's own. As of ULiege regulation, plagiarism is a fraud and is sanctioned with a grade of 0/20 for the course. In particular, giving his/her source code to another student, or copying the source code from another student submission, past or present, is strictly prohibited. Copying from an external source is allowed if the part that is copied is well identified and the source quoted. Note that, in that case, the quoted material will not award any points in the grading. In case of suspicion of fraud, an oral examination will be conducted to determine if the project is indeed the student's work.

Good programming...